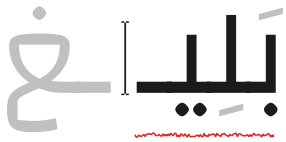
Cairo University
Faculty of Engineering
Department of Computer Engineering



Baligh

Arabic Writing Assistant

A Graduation Project Report Submitted

to

Faculty of Engineering, Cairo University

in Partial Fulfillment of the requirements of the degree

of

Bachelor of Science in Computer Engineering.

Presented by

Amir Anwar Bekhit Awd Kedis

Supervised by

Dr. Ayman AboElhassan

July, 2026

Abstract

Arabic digital writing still suffers from a shortage of integrated tools that can correct Modern Standard Arabic text accurately, respond in real time, and explain their feedback in a way that helps users learn. Existing tools often focus on surface spelling fixes or black-box suggestions without exposing the grammatical reasoning behind the correction. Baligh addresses this problem by providing an Arabic writing assistant that combines grammatical error detection, grammatical error correction, spelling-aware editing, next-word suggestion, and explanatory feedback in a single user-facing system. The project aims to support students, educators, and Arabic content writers by improving writing quality while also making the correction process more understandable and educational.

The implemented system follows a modular pipeline. A preprocessing layer performs normalization, word-boundary handling, segmentation, and morphological analysis and disambiguation. On top of that, Baligh integrates a grammatical error detection engine that fuses rule-based, lexicon-based, and machine-learning detectors, then passes its findings to correction components built around ontology-guided reasoning, dictionary lookup, and text-editing models. In parallel, a next-word suggestion module supports both next-word prediction and word auto-completion. The final system highlights issues, proposes corrections, returns ranked suggestions, and presents grammar-oriented explanations to the user in an interactive workflow.

The project outputs include the preprocessing pipeline, grammatical error detection and correction services, the next-word suggestion subsystem, and a working user application. Testing and validation relied on automated unit and integration tests together with evaluation on established Arabic linguistic resources, including QALB14, QALB15, and ZAEBUC. Overall, Baligh demonstrates that a modular and explainable Arabic writing assistant can combine linguistic processing and machine learning into a practical platform for improving Arabic writing quality.

Contents

List of Figures

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CRF	Conditional Random Field
GEC	Grammatical Error Correction
GED	Grammatical Error Detection
GUI	Graphical User Interface
MSA	Modern Standard Arabic
NLP	Natural Language Processing
NWP	Next-Word Prediction
POS	Part of Speech
QALB	Qatar Arabic Language Bank

Chapter 1: Introduction

1.1. Project Overview

Baligh is an Arabic writing assistant designed to support Modern Standard Arabic writing through explainable correction and real-time assistance. This chapter introduces the motivation behind the project, defines the problem it addresses, summarizes the main outcomes delivered by the project, and outlines the organization of the remainder of the report.

1.2. Individual Role Overview

My individual contribution was centered on the research and implementation of the Grammatical Error Detection (GED) module. I was responsible for studying previous work in Arabic GED and GEC, extracting the most practical design patterns from the literature, and translating them into an explainable multi-detector architecture. This work covered the rule-based detector, lexicon matcher, machine-learning sequence labeler, and the fusion layer that reconciles detector outputs into one final error set.

In addition to the core GED work, I contributed to the product side of Baligh by shaping part of the UI/UX direction, implementing frontend pieces in the web client, and supporting backend integration so that GED results could flow into the editor experience through stable API contracts. These supporting contributions are presented in this report as delivery-enabling work around the main technical contribution.

1.3. Document Organization

This booklet presents my role, the preparation that informed my work, the GED-focused contribution chapter, the supporting frontend and integration work, and finally the main validation results, lessons learned, and future improvements.

Chapter 2: Personal Role and Responsibilities

2.1. Team Responsibilities

My responsibilities during the project combined research, engineering, and delivery support. On the research side, I reviewed Arabic GED, GEC, and writing-assistance literature to understand available datasets, common modeling strategies, and the trade-off between interpretability and coverage. On the implementation side, I owned the main GED architecture and its supporting documentation, notebooks, tests, and evaluation flow. I also participated in making the final system usable by contributing to frontend behavior, UI presentation decisions, and backend integration tasks related to editor-facing analysis results.

2.2. Assigned Tasks

My assigned tasks can be summarized as follows:

- Conduct an extensive literature review on Arabic grammatical error detection and correction, with emphasis on explainable systems and sequence-labeling approaches.
- Design and implement the GED architecture used in Baligh, including rule-based checks, lexicon-backed matching, a machine-learned detector, and a fusion layer.
- Build technical documentation and experimentation notebooks that made the subsystem easier to analyze, evaluate, and extend.
- Contribute to the UI/UX direction of the user-facing product, especially the presentation of correction-oriented interactions.
- Develop selected frontend pieces in the React web client and support backend integration so GED outputs could be consumed consistently by the editor workflow.

2.3. Timeline and Deliverables

Table 2.1: *Major Milestones in My Individual Contribution*

Phase	Main Focus	Deliverables
Research phase	Survey Arabic GED/GEC literature, datasets, and system-design patterns	Research notes, literature synthesis, architecture direction
Rule-based GED phase	Build interpretable detectors on top of preprocessing output	YAML/Python rules, registry, tests, notebooks
Lexicon and ML phase	Extend coverage through curated patterns and sequence labeling	Lexicon matcher, ML detector, training/runtime artifacts, evaluation scripts
Frontend and design phase	Help shape how Baligh appears and behaves in the browser	UI direction, design-system-aligned pages, editor-facing components
Integration and validation phase	Connect detectors to the editor workflow and verify quality	API contract support, integration checks, aggregate evaluation report

Chapter 3: Knowledge Acquisition and Technical Preparation

3.1. Investigated Papers

The literature review focused on Arabic grammatical error detection, grammatical error correction, and practical Arabic writing-assistance systems. The work of Alhafni et al. clarified the value of treating Arabic GED as a sequence-labeling task and highlighted the benefit of passing detection information into later correction stages [1]. Earlier rule-oriented systems such as SAHSOH@QALB-2015 showed that hand-crafted rules still matter in Arabic when interpretability is important [3]. Ontology-driven work on Arabic syntax correction reinforced the value of structured grammatical reasoning and explanation-friendly system design [2]. I also studied preprocessing and morphology-analysis references because robust segmentation, disambiguation, and feature extraction are essential prerequisites for meaningful grammatical analysis.

The research phase was not only descriptive; it directly influenced implementation choices. Rather than selecting a single detector family, I adopted a layered design in which interpretable rules and lexicon signals handle precise, explanation-friendly cases while the machine-learning component improves coverage on broader contexts. This hybrid direction was consistent with the patterns seen in the literature and more suitable for an educational writing assistant than a fully opaque end-to-end model.

3.2. Self-Learning Materials

To implement the project effectively, I relied on a mix of coursework, research papers, library documentation, experimentation notebooks, and framework references. The main self-learning inputs were:

- **College Machine Learning course by Dr. Yahia Zakaria**: strengthened the theoretical background behind the statistical modeling side of the project.
- **College Natural Language course by Dr. Sandra Wahid**: reinforced the language-processing concepts most relevant to Arabic NLP work.
- **Arabic NLP pipeline study**: focused on the preprocessing and morphology-

analysis workflow used in the project, especially segmentation, disambiguation, and morphological feature access.

- **GED modeling study**: covered sequence-labeling concepts, token-level feature engineering, and artifact-based model packaging.
- **Software and integration study**: included FastAPI for service integration, React and Vite for the web client, and Python testing workflows for unit and integration validation.

3.3. Relevance to the Project

This technical preparation shaped the implemented architecture in several direct ways. First, the literature justified keeping symbolic detectors in the loop because Baligh aims to explain as well as detect errors. Second, the sequence-labeling literature supported the use of a learned detector for broader grammatical coverage. Third, Arabic preprocessing references made it clear that morphology-aware input is necessary before higher-level error reasoning can become reliable. Finally, my study of evaluation practice led to a reporting pipeline that measures each GED subsystem individually and then after fusion, allowing the final design to be argued using evidence rather than intuition alone.

Chapter 4: Individual Contributions

4.1. Contribution 1: Research Foundation and GED Implementation

The main technical contribution presented in this booklet is the design and implementation of the Baligh GED subsystem. The objective was to build a detector that is practical for Arabic text, strong enough to cover several error categories, and explainable enough to serve an educational writing tool. Instead of relying on a single model family, I implemented a layered detector architecture that combines symbolic and statistical signals before consolidating them through a fusion stage.

4.1.1. Problem Framing from the Literature

The literature showed three recurring realities about Arabic GED and GEC. First, Arabic is highly morphology-sensitive, so error handling becomes unreliable when preprocessing is shallow or inconsistent. Second, symbolic systems remain useful because many important grammatical and orthographic phenomena still benefit from explicit linguistic checks. Third, learned sequence-labeling approaches improve coverage and fit naturally into multi-class GED pipelines [1–3]. This motivated a hybrid implementation in which different detector families contribute complementary strengths rather than competing as isolated alternatives.

4.1.2. GED Architecture

The implemented GED service receives preprocessed tokens and morphological features, runs several detectors on the same normalized payload, and then resolves conflicts between their outputs. The high-level system context of Baligh is shown in Figure 4.1, while the detector-focused GED architecture appears in Figure 4.2. These figures were generated as part of the project documentation and deliverables rather than drawn specifically for this booklet.

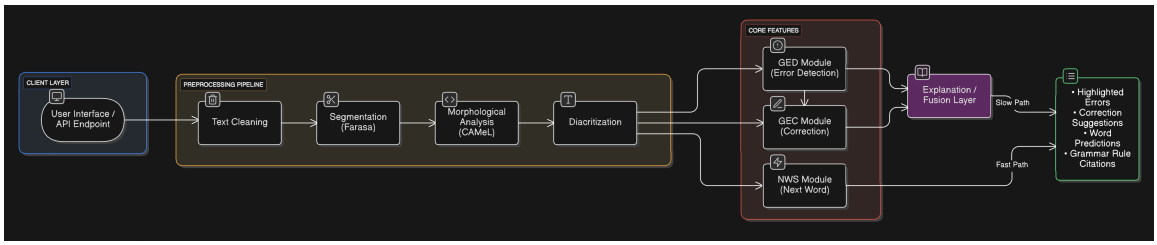


Figure 4.1: *High-level Baligh architecture showing preprocessing, core modules, and user-facing flow.*

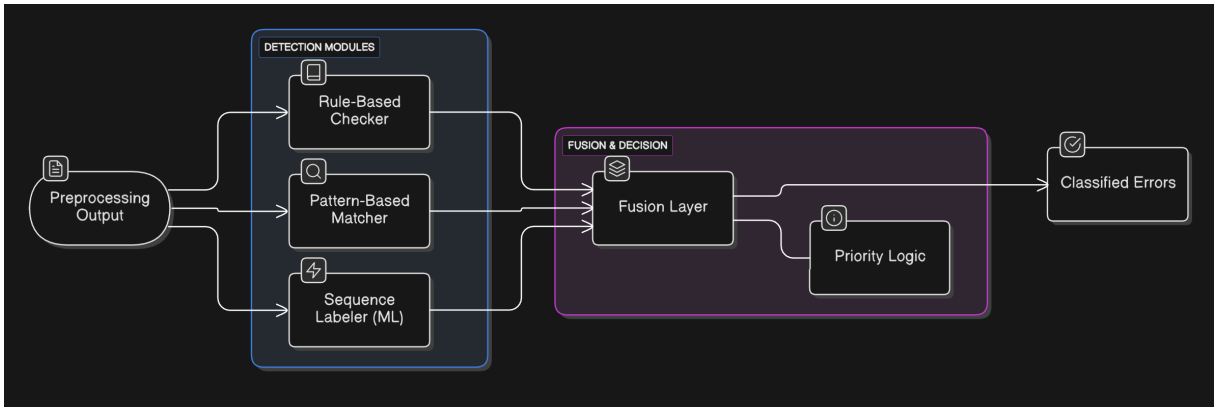


Figure 4.2: *GED detector architecture with rule-based, lexicon-based, and machine-learning detectors followed by fusion and priority logic.*

4.1.3. Rule-Based Subsystem

The rule-based detector was the most interpretable lane in the GED stack. Its main design points were:

- **Hybrid rule hosting**: checks are routed through a registry that supports both declarative YAML rules and registry-based procedural Python rules.
- **Why both formats**: YAML handled compact pattern-driven checks, while Python covered cases that needed richer procedural logic.
- **Authoring workflow**: an agent was fed rule-source material from LanguageTool and other online grammar references to draft rules in our internal format.
- **Quality control**: generated rules were reviewed manually, aligned to our tag schema and preprocessing assumptions, then tested one by one before acceptance.
- **Practical value**: this subsystem was especially useful for high-confidence, explanation-friendly findings that could be stated explicitly and verified deterministically.

4.1.4. Lexicon Subsystem

The lexicon subsystem extended detection beyond purely procedural rules by combining curated token and split/merge patterns with trie-backed lookup and conservative spelling suspicion. Its role was to act as a middle layer between rigid symbolic rules and broader machine-learned predictions.

Main lexicon design points

- **General word lexicon**: used to decide whether a normalized token is a plausible in-vocabulary Arabic word.
- **Named-entity lexicon**: used to protect people, places, organizations, and other entity mentions that may not appear in the general word list.
- **Why both**: the word lexicon supports ordinary spelling lookup, while the entity lexicon reduces false positives on proper nouns and multi-token names.

Table 4.1: *Lexicon Resources Used by the GED Lexicon Detector*

Resource	Size
General word lexicon	9,009,550 entries
Named-entity phrase lexicon	60,833 phrases
Named-entity token lexicon	43,292 tokens

4.1.5. Machine-Learning Subsystem

The machine-learning subsystem was explored through the `ged_ml.ipynb` workflow, where I compared token-memory heuristics, calibrated token memory, linear classifiers, and CRF sequence models before selecting the final detector family. However, the final deployed model was not trained from the notebook directly. It was trained through the dedicated ML training module and exported as the artifact bundle used at runtime. The final formulation treats GED as token-level sequence labeling over Baligh categories, and the final morphology-aware CRF artifact was trained on more than QALB14 alone: its training bundle uses both `QALB14` and `QALB15`. According to the artifact manifest, the kept training data covered 19,706 sentences and 1,063,125 tokens across those two corpora.

Feature engineering

- **Surface features**: normalized token form, token shape, token length, Arabic/digit/punctuation flags, prefixes and suffixes up to length four, sentence-boundary markers, neighboring normalized tokens, sentence-length and position buckets.

- **Orthographic cues**: diacritic presence, repeated-character indicators, definite-article cues, and character n-grams.
- **Morphology features**: POS, UD POS, lexeme, stem, root, pattern, gender, number, person, aspect, voice, mood, state, case, proclitic and enclitic fields.
- **Confidence and context features**: backoff-source indicators, discretized morphology log-probability buckets, and neighboring POS features.

Selected model path

Table 4.2: *ML Detector Selection Summary from* `ged_ml.ipynb`

Role	Model	Threshold	Precision	Recall	F1
Earlier deployment candidate	Surface-only CRF (`crf_surface_v1`)	0.35	0.9080	0.8070	0.8545
Final deployed module	CRF + morphology (`crf_surface_v1_morph_v1`)	0.40	0.9158	0.8332	0.8725

The final runtime detector is therefore the morphology-enhanced CRF, loaded from a model bundle with an inference manifest so that the training-time feature contract stays aligned with deployment. Although the surface-only CRF was an earlier lightweight candidate, the final selected module was `crf_surface_v1_morph_v1` because it delivered the strongest development performance and better category-sensitive signal through the added morphological features.

4.1.6. Fusion Layer

One of the most important parts of my work was the fusion layer, because a multi-detector system is only useful if overlapping outputs can be reconciled consistently. The implemented algorithm first sorts spans, then performs a single left-to-right sweep over the accepted list, applying rules for exact-span duplicates, containment, partial overlap, and tier mismatches. It also normalizes explanation eligibility so that statistical predictions do not claim rule-like explanations that they do not actually possess. Figure 4.3 summarizes this logic using the generated visualization from the GED documentation.

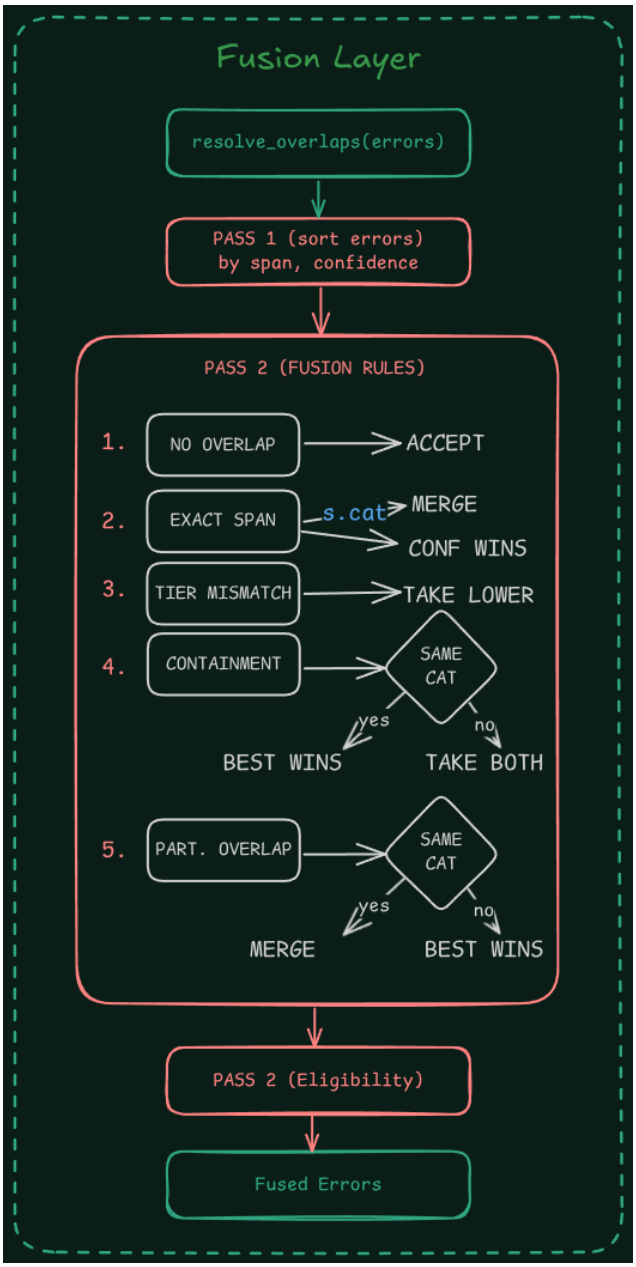


Figure 4.3: *Fusion-layer flow used to resolve overlap conflicts and normalize final GED spans.*

Table 4.3: *Implemented GED Subsystems and Their Roles*

Subsystem	Purpose	Main Strength	My Implementation Role
Rule-based detector	Apply linguistic checks over preprocessed text	Precision and explainability	Implemented detector entry point, rule organization, tests, and supporting notebooks
Lexicon matcher	Detect curated lexical and spelling-oriented issues	Useful bridge between rules and broad statistical prediction	Implemented pattern matching, trie-backed resources, and plausibility gating
Sequence labeler	Predict token-level GED labels from features	Broad contextual coverage	Implemented runtime detector, artifact alignment, and evaluation support
Fusion layer	Merge subsystem outputs into one final result set	Stable multi-detector behavior	Designed and implemented overlap resolution and explanation-eligibility logic

4.1.7. Evaluation Summary

The implemented evaluation pipeline measures each subsystem independently and then after fusion across several corpora, including QALB14, QALB15 L1, QALB15 L2, and ZAEBUC. Aggregate binary results from the generated report show that the symbolic lanes are precise but narrower in coverage, while the learned and fused systems provide much stronger end-to-end performance. The aggregate binary F1 scores were approximately 0.277 for the rule-based detector, 0.230 for the lexicon matcher, 0.808 for the sequence labeler, and 0.819 for the fused system.

These numbers should also be interpreted with care because the error categories are not evenly distributed across the evaluation datasets. In practice, the datasets highlight models that perform well on the more frequent tags, which naturally favors the sequence labeler. However, the rule-based and lexicon-based lanes still catch error types that the sequence labeler may miss, especially categories that are less common in the training and evaluation sets. For that reason, their value is not fully reflected by aggregate binary scores alone, and the fused gain is important because it shows that these symbolic detectors still contribute useful coverage beyond the dominant dataset patterns.

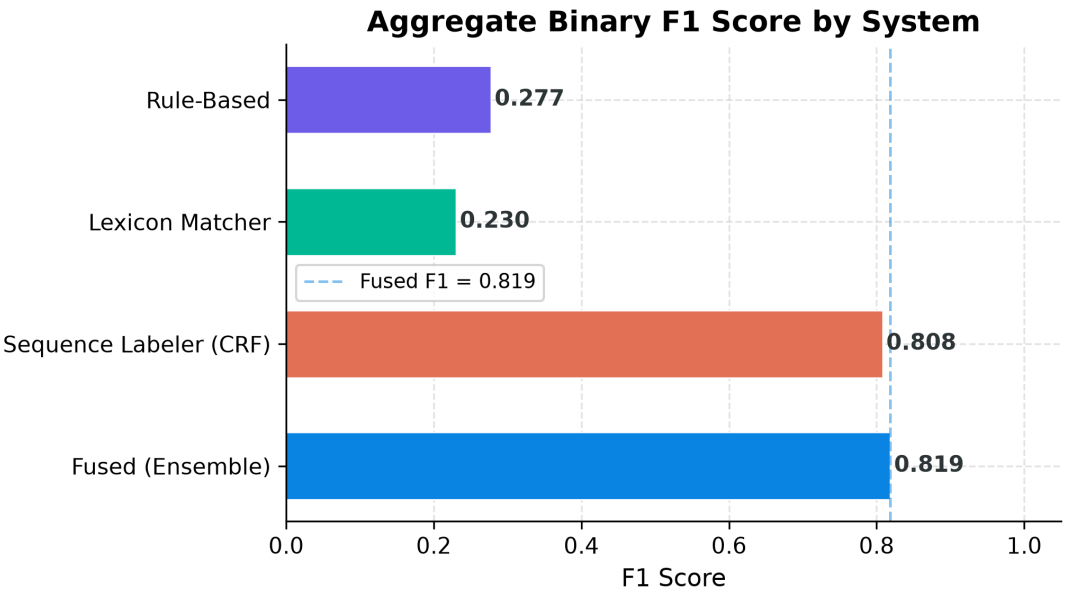


Figure 4.4: *Aggregate binary GED F1 score by subsystem from the generated evaluation figures.*

Table 4.4: *Aggregate Binary GED Performance*

System	Precision	Recall	F1
Rule-Based	0.685	0.174	0.277
Lexicon Matcher	0.892	0.132	0.230
Sequence Labeler	0.913	0.725	0.808
Fused	0.905	0.747	0.819

4.1.8. GED Output in the GUI

Because Baligh is ultimately a user-facing writing assistant, the detector output had to be understandable in the editor rather than only measurable in an evaluation script. Figure 4.5 therefore shows two real GUI captures: one emphasizing highlighted detections inside the editor text, and one showing the issue card and explanation-oriented presentation attached to a detected span.

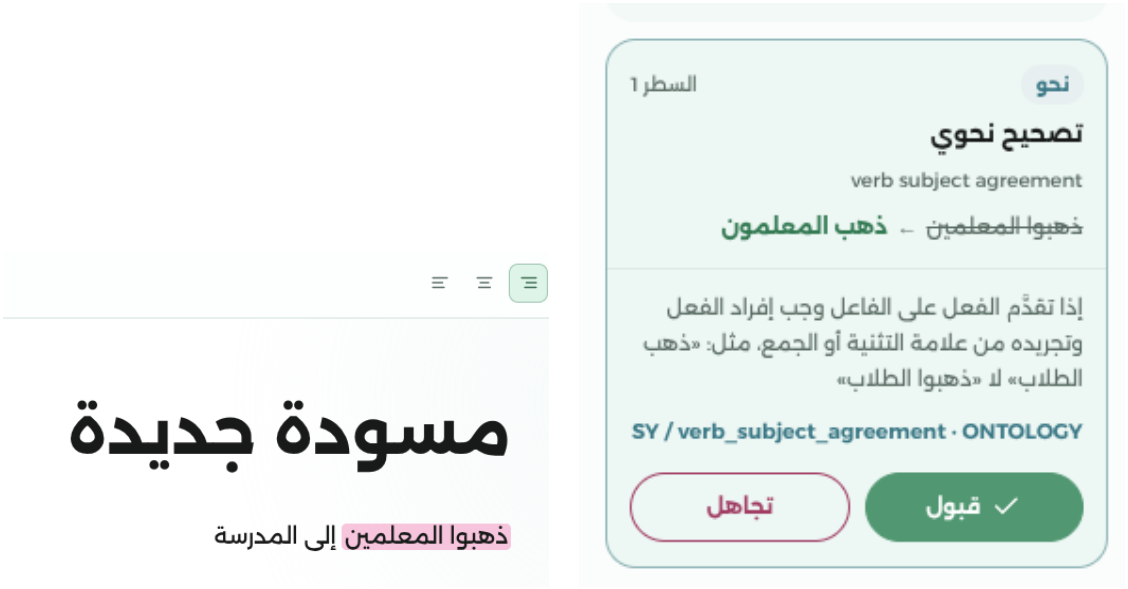


Figure 4.5: *GED feedback inside the Baligh editor: highlighted detections in the writing view and an issue card showing explanation-oriented feedback for a detected error.*

4.2. Contribution 2: Productization Through Frontend, UI Direction, and Integration Support

While GED was my primary technical ownership area, I also contributed to making the subsystem usable inside the product. This work was not presented as a separate research contribution; rather, it helped turn backend analysis into a coherent editor workflow.

4.2.1. UI/UX Direction

My UI/UX contribution focused on clarity rather than visual novelty alone. Since Baligh is intended to support writing improvement, the interface needed to make corrections approachable and readable for Arabic users. This influenced the direction of the editor flow, the presentation of highlighted issues, and the general preference for an Arabic-first, learning-oriented experience over a noisy or overloaded correction dashboard. The goal was to ensure that detector outputs felt like guided feedback instead of raw diagnostic logs.

4.2.2. Frontend Implementation

The web client provided a concrete path for exposing Baligh's capabilities to users. My contribution here included participation in frontend development around the landing ex-

perience, editor-facing interaction, and reusable design-system-aligned components. The repo reflects this through the React/Vite client structure, themed assets, UI components, and test coverage for page behavior and controller logic. Figure 4.6 illustrates this product layer through the public landing view and the editor workspace where analysis results are consumed. Although this was not the flagship contribution of my booklet, it was important because it created the presentation layer that transformed GED results into something inspectable and actionable.

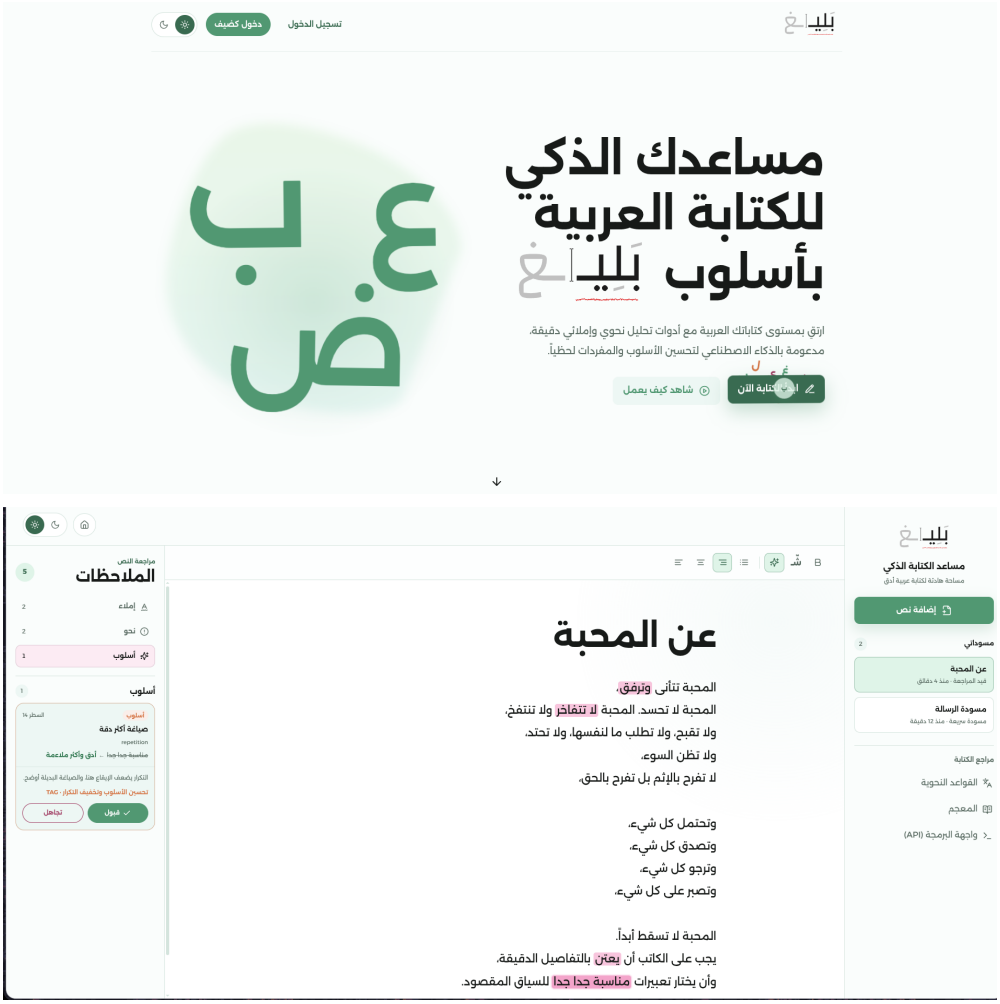


Figure 4.6: *Frontend-facing product surfaces: the Baligh landing page and the editor workspace used to present analysis results to the user.*

4.2.3. Integration and Backend Support

I also supported the backend and integration work needed to expose analysis results to the editor through stable service contracts. This helped ensure that findings, corrections, and suggestions were normalized into frontend-ready structures and fit the final editor workflow.

4.3. Testing and Validation

My contribution was validated through three main layers: unit tests for GED components such as rules, lexicon logic, and fusion; experimentation notebooks for inspecting preprocessing and detector behavior on real Arabic examples; and formal multi-corpus evaluation of the rule-based, lexicon, sequence-labeling, and fused systems. The surrounding product work was also checked through frontend tests, end-to-end flows, and backend contract-level validation.

Chapter 5: Challenges, Lessons Learned, and Future Work

5.1. Faced Challenges

The main challenges were acquiring suitable Arabic GED data, since high-quality labeled resources are limited, and building the rule base itself. Writing rules required both finding reliable grammar sources and translating them into forms that could be implemented, reviewed, and tested consistently.

5.2. Lessons Learned

- Research becomes valuable when it directly shapes implementation and evaluation decisions.
- Experimental NLP work becomes easier to maintain when it is supported by tests, artifacts, and clear interfaces.
- Product value depends not only on detection quality, but also on presenting model output clearly to users.
- The benefit of good communication and documentation.

5.3. Future Enhancements

Several improvements could extend this work in future iterations. The rule base can be broadened to cover more grammatical constructions and edge cases, especially for syntax and morphology. The machine-learning detector can benefit from richer training data, additional feature refinement, and stronger handling of underrepresented error categories. The interaction between GED and later correction stages can also become tighter so that detection evidence more directly shapes candidate generation and ranking.

References

[1] B. Alhafni, N. Inoue, and N. Habash, “Advancements in Arabic Grammatical Error Correction: The Promise of Large Language Models and a Two-Stage System,” in *Proceedings of ArabicNLP*, 2023.

[2] M. Moukrim, A. Namir, and M. Boulaknadel, “An Innovative Approach for Autocorrecting Arabic Syntactic Errors Based on Ontology,” *International Journal of Advanced Computer Science and Applications*, 2021.

[3] W. Zaghouani, N. Habash, O. Obeid, H. Bouamor, and S. Khalifa, “SAHSOH@QALB-2015 Shared Task: A Hybrid Arabic Spelling and Grammar Correction System,” in *Proceedings of the Second Workshop on Arabic Natural Language Processing*, 2015.

[4] R. Belkebir and N. Habash, “Automatic Error Type Annotation for Arabic,” in *Proceedings of the 6th Arabic Natural Language Processing Workshop*, 2021.

Chapter A: GED Rules Reference

This appendix summarizes the rule inventory that supports the implemented GED subsystem. The rule-based catalog is documented in `baligh/docs/ged/rules.md`, while the split and merge lexicon patterns are maintained in `src/services/ged/detectors/lexicon/resou`
The goal of this appendix is to keep the booklet aligned with the actual implementation rather than presenting a rewritten or disconnected list.

A.1. Rule-Based GED Catalog

A.1.1. Orthography Rules

Rule ID	Description
OT_HAMZA_PREP	حروف الجر والربط وبعض الأدوات التي أصلها بهمزة قطع تكتب بالهمزة لا بالألف المجردة، مثل: إلى، أو، إذا، أن، إن.
OT_ALIF_MAQSURA_ALA	الصواب «على» لا «علي».
OT_ALIF_MAQSURA_HATTA	الصواب «حتى» لا «حتي».
OT_TANWIN_NASB_ON_ALIF	تنوين النصب يكتب على الحرف الذي قبل الألف لا على الألف نفسها.
OT_IDGHAM_AN_MA	الأفصح إدغام «عن ما» في «عمّا» عندما تليها جملة فعلية.
OT_IDGHAM_MIN_MA	الأفصح إدغام «من ما» في «ممّا» عندما تليها جملة فعلية.
OT_TA_MARBUTA_NOUN	وجوب كتابة التاء المربوطة «ة» في أواخر الأسماء المؤنثة بدل الهاء «ه».
OT_TA_MARBUTA_ADJ	وجوب كتابة التاء المربوطة «ة» في أواخر الصفات المؤنثة بدل الهاء «ه».
OT_TA_MARBUTA_NOUN_PROP	وجوب كتابة التاء المربوطة «ة» في أواخر الأعلام المؤنثة بدل الهاء «ه»، مثل: مكة، فاطمة.
OT_INNA_AFTER_QAWL	همزة «إن» تُكسر وجوباً بعد القول.
OT_IDHA_HAMZA	«إذا» تُكتب بهمزة قطع تحت الألف.
OT_ARWAH_HAMZA	كلمة «أرواح» جمع تكسير تُكتب بهمزة قطع على الألف.
OT_AQAL_HAMZA	أفعل التفضيل «أقل» يُكتب بهمزة قطع على الألف.

A.1.2. Punctuation Rules

Rule ID	Description
PC_SPACE_BEFORE_PUNC	وجوب اتصال علامات الترقيم العربية مباشرة بالكلمة التي تسبقها دون فاصل أو مسافة.
PC_LATIN_COMMA_ARABIC	في السياق العربي تستعمل الفاصلة العربية «،» لا الفاصلة اللاتينية.
PC_LATIN_QUESTION_ARABIC	في السياق العربي تستعمل علامة الاستفهام العربية «؟» لا العلامة اللاتينية.
PC_LATIN_SEMICOLON_ARABIC	في السياق العربي تستعمل الفاصلة المنقوطة العربية «؛» لا العلامة اللاتينية المناظرة.

A.1.3. Syntax Rules

Rule ID	Description
SY_DEM_HADHANI_FEM	الصواب «هاتان» مع الاسم المثنى المؤنث لا «هذان».
SY_DEM_HATANI_MASC	الصواب «هذان» مع الاسم المثنى المذكر لا «هاتان».
SY_DEM_HADHAYNI_FEM	الصواب «هاتين» مع الاسم المثنى المؤنث لا «هذين».
SY_DEM_HATAYNI_MASC	الصواب «هذين» مع الاسم المثنى المذكر لا «هاتين».
SY_DEM_HADHANI_CASE_OBLIQUE_NOUN	إذا جاء بعد «هذان» اسم مثنى مذكر غير مرفوع فهناك خلل في المطابقة الإعرابية.
SY_DEM_HADHAYNI_CASE_NOM_NOUN	إذا جاء بعد «هذين» اسم مثنى مذكر مرفوع فهناك خلل في المطابقة الإعرابية.
SY_DEM_PREP_HADHAYNI_CASE_NOM_NOUN	بعد حرف الجر يكون الاسم بعد «هذين» مجرورًا لا مرفوعًا.
SY_DEM_HATANI_CASE_OBLIQUE_NOUN	إذا جاء بعد «هاتان» اسم مثنى مؤنث غير مرفوع فهناك خلل في المطابقة الإعرابية.
SY_DEM_HATAYNI_CASE_NOM_NOUN	إذا جاء بعد «هاتين» اسم مثنى مؤنث مرفوع فهناك خلل في المطابقة الإعرابية.
SY_DEM_PREP_HATAYNI_CASE_NOM_NOUN	بعد حرف الجر يكون الاسم بعد «هاتين» مجرورًا لا مرفوعًا.
SY_LAM_JUSSIVE	الفعل المضارع بعد «لم» يجب أن يكون مجزومًا.
SY_LAMMA_JUSSIVE	الفعل المضارع بعد «لما» يجب أن يكون مجزومًا.
SY_LA_NAHIYA_JUSSIVE	الفعل المضارع بعد «لا» الناهية يجب أن يكون مجزومًا.
SY_LA_NAFIYA_NOT_JUSSIVE	«لا» النافية لا تجزم الفعل المضارع.

Rule ID	Description
SY_INNA_SISTERS_DUAL_ACCUSATIVE	أخوات إن تنصب الاسم، والمثنى بعدها يكون منصوبًا بالياء لا مرفوعًا.
SY_VERB_SUBJECT_VSO	وجوب إفراد الفعل وتجريده من علامات التثنية والجمع إذا تقدم على الفاعل الظاهر في الجملة الفعلية.
SY_NOUN_ADJ_DEFINITENESS	وجوب مطابقة النعت للمنعوت في التعريف والتنكير.
SY_DEMONSTRATIVE_NOUN_GENDER	وجوب مطابقة اسم الإشارة للاسم الذي بعده في التذكير والتأنيث، مثل: هذا البطل، هذه السلامة.
SY_RELATIVE_PRONOUN_GENDER	وجوب مطابقة الاسم الموصول للاسم السابق له في التذكير والتأنيث، مثل: القول الذي، الوشاية التي.
SY_PREP_DUAL_CASE	وجوب جر الاسم المثنى بعد حرف الجر، مثل: في الكتابين لا في الكتابان.
SY_PREP_SOUND_MASC_PLURAL_CASE	وجوب جر جمع المذكر السالم بعد حرف الجر، مثل: مع المسافرين لا مع المسافرون.
SY_MAMNOO3_SARF_TANWEEN_NASB	الممنوع من الصرف لا يُنوَّن.
SY_KAMA_ANNA	الصواب «كما أن» بدون الواو الزائدة.
SY_LA_SIYYAMA_WA	الصواب «لا سيما» دون واو بعدها.
SY_AS_KA	استخدام الكاف للصفة استخدام أجنبي، والأصح النصب على الحالية.
SY_LA_PAST_TENSE	عند نفي الفعل الماضي يُنفى بـ «ما» أو «لم» مع المضارع، ولا يصح استخدام «لا» إلا إذا تكررت أو كانت للدعاء.
SY_MUBTADA_NAKIRA_PREP	المبتدأ النكرة يؤخر وجوباً إذا كان الخبر شبه جملة من جار ومجرور.
SY_QAAMA_BI	تعبير «القيام بـ» ركيك يستحسن تجنبه واستعمال الفعل مباشرة.
SY_HAWLA_FI	يفضل استعمال حرف الجر «في» بدلاً من «حول» في هذا السياق.
SY_RAGHBA_LI	يُقال: «رغبة في» وليس «رغبة لـ».
SY_AATIL_AN	الأفصح أن يقال: «عاطل من العمل» أو «متعطل».
SY_TA2MEEN_DHIDDA	الأفصح «تأمين من» أو «مناعة من» بدلاً من «ضد».

A.1.4. Semantics Rules

Rule ID	Description
SE_DECADES_IYAT	يفضل في العقود الكتابة بصيغة «ـينيات» لا «ـينات»، مثل: الثلاثينيات.
SE_MOAKHARAN	يفضل تجنب «مؤخرا» بهذا المعنى، والأفصح: حديثًا أو قريبًا.
SE_MUTAAKID	يفضل استعمال «متحقق» أو «متيقن» بدل «متأكد».

Rule ID	Description
SE_DHATA	يفضل «ذواتا» بدل «ذاتا» في هذا الاستعمال.
SE_KHATIR	يفضل في هذا الاستعمال: شديد الخطر أو محفوف بالخطر بدل «خطير».
SE_KHAMMARA	«الخمارة» بائعة الخمر، ويستحسن للمكان: «مخمرة» أو «حانة».
SE_INDHAHALA	الفعل الأفصح: «ذهل» لا «انذهل».
SE_BIAKMALIHI	يفضل «كله» أو «جميعه» أو «برمته» بدل «بأكمله».
SE_TAHAMMAMA	يفضل استعمال «استحم» بدل «تحمم».
SE_TASHKILU	يستحسن الاستغناء عن «تشكل» في هذا الاستعمال أو استبدالها بـ«هي».
SE_TASAMAMA	الفعل المضعف هنا يدغم، والأفصح: «تصامّ» لا «تصامم».
SE_TATMIN	الأفصح: «طمأنة» لا «تطمين».
SE_TA3KISU	يفضل في هذا السياق «تظهر» أو «تفضح» بدل «تعكس».
SE_JANOOBI	في الظرفية المكانية يفضل «جنوبَ» بدل «جنوبي».
SE_KHESISAN	يفضل «خِصِّيصَى» أو «خاصًا» بدل «خصيصا».
SE_KHALOOQ	يفضل في الوصف هنا: «حسن الخلق» بدل «خلوق».
SE_RAGHMA	يفضل «على الرغم» أو «بالرغم» أو «على» أو «مع» بدل «رغم».
SE_RAFAH	المستعمل هنا «رفات» لا «رفاة».
SE_SHAWYAN	الأفصح في مصدر «شوى»: «شَيًّا» لا «شويا».
SE_ARAYA	يفضل في هذا المعنى «عريانـون» لا «عرايا».
SE_LIWAHDIHI	الأفصح: «وحده» بدل «لوحده».
SE_MAHALAT	جمع «محل» في هذا الاستعمال هو «محالّ» لا «محلات».
SE_NAFOUKH	الأفصح: «يافوخ» بدل «نافوخ».
SE_NASHET	في الوصف يفضل «نشيط» أو «ناشط» بدل «نشط».
SE_WALLATI	يستحسن حذف الواو من «والتي» في هذا الربط.
SE_WALLADHI	يستحسن حذف الواو من «والذي» في هذا الربط.
SE_ITTILA3	الأفصح: «اطّلاع» بدل «إطلاع».
SE_IDHTARADA	الأفصح: «اطرّد» بدل «اضطرد».
SE_MUJBAA	الأفصح: «مجبية» بدل «مجباة».
SE_MOUSOUD	الأفصح: «موصَد» بدل «موصود».
SE_MUASHIRAT	يستحسن في هذا المعنى: إشارات أو علامات أو شواهد أو دلائل بدل «مؤشرات».

Rule ID	Description
SE_MISHWAR	يستحسن تجنب «مشوار» بهذا المعنى، والأفصح: طريق أو مسار أو نزهة.
SE_MACHINE	الأفصح: «مكنة» بدل «ماكينة».
SE_IMKANIYAT	يفضل «إمكانات» بدل «إمكانيات».
SE_TAWAJUD	يفضل في هذا الاستعمال «وجد» بدل «تواجد».
SE_MUSBAQAN	يفضل «مقدمًا» أو «سلفًا» أو «قبلًا» بدل «مسبقا».
SE_WABITTALI	«وبالتالي» تعبير ركيك في هذا السياق، ويستحسن استبداله ببدائل أفصح.
SE_BIMATHABATI	يفضل «بمنزلة» أو «تقوم مقام» بدل «بمثابة».
SE_ISTIBYAN	يفضل «استبانة» بدل «استبيان».
SE_AKHISSAI	يفضل «اختصاصي» بدل «أخصائي».
SE_TOQOS	يفضل «شعائر» بدل «طقوس» في هذا الاستعمال.
SE_BIHASABI	يستحسن تجنب «بحسب» في هذا الاستعمال واستبدالها ببدائل أفصح.
SE_LISALIHIKA	يفضل «لمصلحتك» بدل «لصالحك».
SE_LISALIHI	يفضل «لمصلحة» بدل «لصالح».
SE_I3TABAR	يفضل في هذا الاستعمال «عدّ» بدل «اعتبر».
SE_BURHA	يفضل «هنيهة» بدل «برهة» في هذا السياق.
SE_AAWINA	يفضل «أوان» بدل «آونة».
SE_BAWASIL	في هذا الاستعمال يفضل «بسلاء» أو «باسلون» بدل «بواسل».
SE_MUSTAHTIR	يفضل «مستهين» أو «مستخفّ» بدل «مستهتر» في هذا الاستعمال.
SE_SUWAH	يفضل «سياح» بدل «سواح».
SE_KAKULL	«ككل» ترجمة ركيكة، ويستحسن «كليًا».

A.2. Lexicon Split and Merge Patterns

The lexicon matcher supplements the rule-based system with exact curated patterns. In this booklet I explicitly list the split and merge entries because they form an important bridge between pure spelling normalization and grammar-aware error detection.

A.2.1. Split Patterns

Pattern ID	Wrong Surface Form	Correct Form	Purpose
LEX_SPLIT_LAKIN	لا كن	لكن	Common split correction
LEX_SPLIT_HAYTHU_MA	حيث ما	حيثما	Conditional/relative split correction
LEX_SPLIT_KAY_LA	كي لا	كيلا	Common split correction
LEX_SPLIT_BAADAMA	بعد ما	بعدما	Common split correction
LEX_SPLIT_LIMAN	ل من	لمن	Clitic attachment correction
LEX_SPLIT_FI_MA	في ما	فيما	Common split correction
LEX_SPLIT_LIMADHA	ل ماذا	لماذا	Interrogative clitic correction
LEX_SPLIT_INDA_MA	عند ما	عندما	Common split correction
LEX_SPLIT_KULLAMA	كل ما	كلما	Conditional/time expression correction
LEX_SPLIT_RUBBA_MA	رب ما	ربما	Common split correction

A.2.2. Merge Patterns

Pattern ID	Wrong Token	Correct Tokens	Purpose
LEX_MERGE_IN_SHA_ALLAH	انشاءالله	إن شاء الله	Common merge correction
LEX_MERGE_MASHA_ALLAH	ماشاءالله	ما شاء الله	Common merge correction
LEX_MERGE_ALHAMD_LILLAH	الحمدلله	الحمد لله	Common merge correction

Pattern ID	Wrong Token	Correct Tokens	Purpose
LEX_MERGE_SUBHAN_ALLAH	سبحاناالله	سبحان الله	Common merge correction
LEX_MERGE_LA_SIYYAMA	لاسيما	لا سيما	Common merge correction
LEX_MERGE_JAZAK_ALLAH	جزاكالله	جزاك الله	Common merge correction
LEX_MERGE_TAWAKKAL_ALLAH	توكلتعلىالله	توكلت على الله	Common merge correction
LEX_MERGE_ASTAGHFIR_ALLAH	استغفرالله	أستغفر الله	Common merge correction
LEX_MERGE_ALLAHU_AKBAR	اللهاكبر	الله أكبر	Common merge correction
LEX_MERGE_BISMILLAH	بسماللهالرحمنالرحيم	بسم الله الرحمن الرحيم	Common merge correction